SEMINAR SUMMARY: FEBRUARY 17, 2026 - MARCO ALDI NP PROBLEMS AND THEIR QUANTUM ANALOGUES

ROB TOMPKINS

Anything that can be calculated on today's computers could have been calculated using mechanical machine. An example of a particularly sophisticated calculating machine was established by Babbage. In fact anything calculable with one of the most sophisticated computers today (quantum aside) could be worked out by hand in a step-by-step process with boundaries given for input and a sufficient set of instructions. These mechanical calculators or computers as we call them, are particularly good at solving a particular set of problems. These are the easy problems with which we are familiar, alphabetizing a list, finding the value of an element in a list, merging two lists, and many more. Though things become difficult when we are asked if an arbitrary collection of integers contains a subset that sums to zero. Moreover, it is even harder to find that subset. Note, there are a collection of problems that are essentially analogous this problem about finding a subset of integers that sums to zero.

In the early 1970's, 1971 to be exact, Stephen Cook and Leonid Levin independently went about attempting to formally define the characteristic to a problem that lands it in one of these two categories. The easier first collection of problems are those that given an input of length n can be solved in $n^k + a$ steps with $a, k \in \mathbb{N}$. These problems we call *polynomial time* problems because they can be solved in $\mathcal{O}(n^k)$ steps the problem space $\mathbf{P}$.

Then there are problems that are known to take a substantial amount of time, for example exhaustive search for a directory in a large un-indexed directory tree. These problems are known to take $\mathcal{O}(2^n)$ steps, or *exponential time.*

Now, these inbetween difficulties problems. How are we to classify them. Originally, we were concerned with polynomial time algorithms over a Non-deterministic Turing Machine. Where a Non-deterministic Turing Machine is one that given a tape position and a state, one has multiple next states into which to move, which leaves us with a relatively vague yet faster path to our solution. Eventually people became impatient with the vagueness of this definition, and the following definition arose.

Definition 1. (The class $\mathbf{NP}$) A language $L \subseteq \{0,1\}^*$ (the kleene closure over $\{0,1\}$) is in $\mathbf{NP}$ provided there exists a polynomial $p : \mathbb{N} \to \mathbb{N}$ and a polynomial-time turing machine M

Date: February 19, 2026.

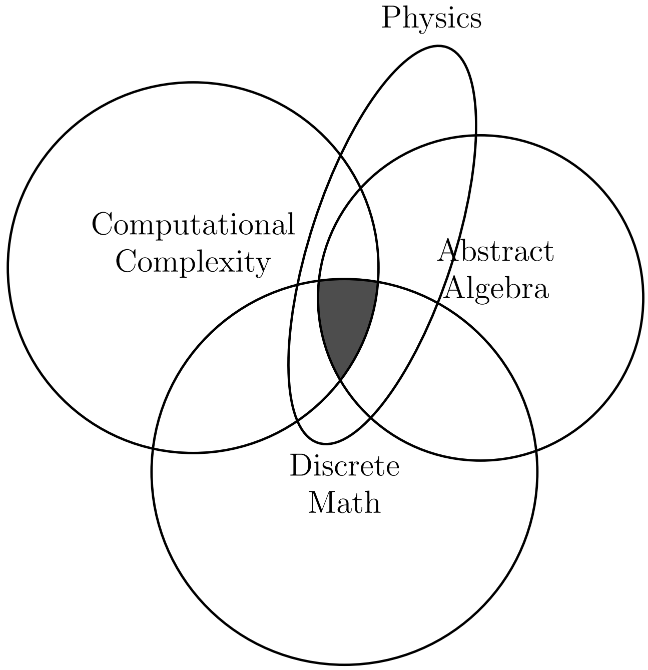

FIGURE 1. Fields involved in making progress on **NP**-hard problem research

(called the *verifier* of L) such that for every $x \in \{0,1\}^*$

$$x \in L \Leftrightarrow \exists u \in \{0,1\}^{p(|x|)} \quad \text{s.t.} \quad M(x,u) = 1.$$

If $x \in L$ and $u \in \{0,1\}^{p(|x|)}$ satisfy $M(x,u) = 1$, then we call u a *certificate* for x (with respect to the language L and the machine M).

Question. Does $\mathbf{P} = \mathbf{NP}$?

Over the years multiple different fields have all made their own progress on the problem. Figure 1 gives us a venn-diagram of some of the more important fields involved in making progress on **NP**-hard problems, whether **P** and **NP** are the same set of problems.

Now that we have the class of **NP**-hard problems, it becomes important to have an example of an **NP**-hard problem. Now we earlier mentioned the integer subset sum problem, but this was not the original problem proven to be NP-hard. That was the boolean satisfaction problem, "SAT". SAT is defined by considering a list of variables $s_0, s_1, \ldots, s_{n-1} \in \{0,1\}$ and a function $f : \{0,1\}^n \to \{0,1\}$, find the answer to $f(s_0, s_1, \ldots, s_{n-1}) = 0$ (or 1). We can think of f as an arbitrary list of and's $\wedge$ and or's $\vee$